



Instituto Infnet

# Git e GitHub: o essencial

**Projeto de Bloco: Inteligência Artificial Aplicada**

Aula 2 · complemento (~20 min)

Prof. Renan Santos

# Por que versionar o projeto?

*Git é um sistema de controle de versão: guarda o histórico do projeto e permite colaborar.*

- **Histórico: cada versão salva é um commit**

Você volta a qualquer ponto e compara duas versões.

Fim do 'app\_v2\_final\_agora\_vai.py' na pasta.

- **Backup fora do seu computador**

HD queimou, notebook sumiu: o projeto continua no GitHub.

E você abre o mesmo projeto de outra máquina.

- **Colaboração sem sobrescrever ninguém**

Duas pessoas trabalham ao mesmo tempo e o Git integra depois.

É assim que o painel vira trabalho de equipe nas próximas etapas.

► **No TP1, o repositório mostra o seu PROCESSO — não só o resultado final.**

# Repositório local × repositório remoto

*São dois repositórios separados — e nada sincroniza sozinho entre eles.*

- **Local: a pasta do projeto + o diretório oculto .git**  
O .git guarda todo o histórico. Nunca edite nada lá dentro.  
Some o .git, some o histórico (os arquivos ficam).
- **Remoto: hospedado num serviço (GitHub, GitLab, Bitbucket)**  
É a cópia na nuvem .  
Pode ser público (todos veem) ou privado (só quem você autorizar).
- **Nenhuma atualização é automática**  
Nada sobe nem desce sem você rodar um comando (push/pull).  
Salvar o arquivo ≠ fazer commit ≠ publicar no GitHub.

► **Salvou ≠ commitou ≠ publicou. São três passos distintos e explícitos.**

# As quatro áreas do Git

Working directory → staging area → commit history (no repositório local) → repositório remoto.

## As áreas do Git — e como um arquivo caminha até o GitHub



# O ciclo do dia a dia

terminal

```
git status                # o que mudou? o que está no staging?
git add app.py src/data_access.py  # escolho o que entra no commit
git commit -m "coleta via API do IBGE"  # salva a versão (no repo LOCAL)
git log --oneline          # os commits, do mais novo ao antigo
git push                   # envia os commits para o GitHub

# 'commit early, commit often': no começo, muitos commits > poucos
```

# Do zero ao GitHub, em 3 passos

terminal

```
git config --global user.name "Seu Nome"          # 1x por computador
git config --global user.email "voce@email.com"    # idem

# 1) criar o repositório local
git init -b main
git add -A
git commit -m "primeira versão do painel"

# 2) conectar ao remoto (crie no GitHub um repositório VAZIO, sem README)
git remote add origin https://github.com/voce/painel-ods.git

# 3) enviar (-u define o padrão: depois basta 'git push')
git push -u origin main
```

# O que NÃO entra no repositório

*Versionar é escolher: código e documentos, sim; segredos e lixo, nunca.*

- **Segredos: chaves de API, .env, senhas**  
Uma vez commitado, fica no histórico mesmo se você apagar depois.
- **Ambiente e cache: .venv/, \_\_pycache\_\_/, .ipynb\_checkpoints/**  
São recriáveis por um comando e só poluem o histórico.
- **Dados brutos pesados**  
Versione o CÓDIGO que coleta, não o CSV de 200 MB — exceto uma amostra.
- **Como se faz: o arquivo .gitignore na raiz**  
Uma linha por padrão a ignorar; o Git passa a fingir que eles não existem.

► **Regra prática: se dá para recriar com um comando, não versiona. Segredo, jamais.**

# Trabalhando em duas máquinas (e com o Colab)

*O mesmo projeto na faculdade, em casa e no Google Colab — sem pendrive.*

- **git clone <URL>**

Baixa o repositório remoto e cria um repositório local na outra máquina.  
É a segunda forma de começar um projeto (a outra é git init).

- **git pull antes de começar, git push ao terminar**

pull traz o que mudou no remoto; push envia o que você fez.  
Esse hábito evita a maioria dos conflitos.

- **O Colab abre notebooks direto do GitHub**

É o que o badge 'Open in Colab' faz: lê o .ipynb do repositório.  
Repositório privado → é preciso autorizar o Colab a acessar sua conta.

► **pull no começo, push no fim: o repositório é o que sincroniza os seus ambientes.**